KCPC Week 5 Papers
Graphs II: Dijkstra and 0–1 BFS

King's Competitive Programming Club

An annotated guide to the original research behind Workshop 5, which introduces shortest-path algorithms in weighted graphs (Dijkstra's algorithm and 0–1 BFS) and situates them in their broader theoretical and historical context. Look at week 4 content for the foundations. For a gentler first pass, use *Week 5 Reading* as these are primary sources and not introductory tutorials.

I. Foundational paper

[1] Dijkstra, E. W. (1959)

A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269–271. https://doi.org/10.1007/BF01386390
Where / why: The whole note is three pages; the shortest-path development is "problem 2", pp. 269–271. This introduces Dijkstra's algorithm, the primary shortest-path method taught in the workshop for graphs with non-negative edge weights, and the source of the slides' settling invariant.

II. Structural and data-structure improvements

[2] Dial, R. B. (1969)

Algorithm 360: Shortest-path forest with topological ordering. Communications of the ACM, 12(11), 632–633. https://doi.org/10.1145/363269.363610
Where / why: The full note, pp. 632–633, introduces bucket-based shortest-path methods for small integer weights, providing the theoretical basis for the 0–1 BFS optimisation (slides: two adjacent distance levels).

[3] Fredman, M. L., & Tarjan, R. E. (1984)

Fibonacci heaps and their uses in improved network optimization algorithms. Journal of the ACM, 34(3), 596–615. https://doi.org/10.1145/28869.28874
Where / why: Introduces the Fibonacci heap; the network-optimisation applications give Dijkstra's algorithm its textbook $O(m + n \log n)$ bound. This shows how priority queue design impacts the efficiency of Dijkstra's algorithm; our workshops use the simpler lazy binary heap.

[4] Johnson, D. B. (1977)

Efficient algorithms for shortest paths in sparse networks. Journal of the ACM, 24(1), 1–13. https://doi.org/10.1145/321992.321993
Where / why: Demonstrates extensions and optimisations of Dijkstra's algorithm for sparse graphs, the regime in which competitive programming problems almost always live.

III. Advanced and modern developments

[5] Thorup, M. (1999)

Undirected single-source shortest paths with positive integer weights in linear time. Journal of the ACM, 46(3), 362–394. https://doi.org/10.1145/316542.316548
Where / why: Proves that single-source shortest paths can be solved in linear time under restricted (undirected, positive integer) weight conditions, bypassing comparison-based priority queues entirely.

[6] Duan, R., Mao, J., Mao, X., Shu, X., & Yin, L. (2025)

Breaking the sorting barrier for directed single-source shortest paths. Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC 2025), 36–44. https://doi.org/10.1145/3717823.3718179 (also arXiv:2504.17033)
Where / why: Gives a deterministic $O(m \log^{2/3} n)$-time directed SSSP algorithm, the first to break Dijkstra's $O(m+n \log n)$ bound on sparse graphs when only distances (not their ordering) are required.

[7] Duan, R., Mao, X., Shu, X., & Yin, L. (2026)

A faster directed single-source shortest path algorithm. arXiv:2602.07868. https://arxiv.org/abs/2602.07868
Where / why: Sharpens [6] to $O(m\sqrt{\log n} + \sqrt{mn \log n \log\log n})$. Included to show that single-source shortest paths remain an active theoretical topic: even classical problems continue to admit structural improvements under refined models.

Summary

Together, these papers trace the development of single-source shortest-path algorithms: from Dijkstra's original method [1], through bucket, heap and sparse-graph optimisations [2–4], to linear-time integer methods and the recent directed-graph breakthroughs [5–7]. For implementation-level expositions with page references, see *Week 5 Reading*.